

- 강의명: CSCI E-103: Introduction to the Intellectual Enterprises of CS
- 주차: Lecture 9: MLOps와 데이터의 중요성
- 교수명: UIUC Faculty
- 목적: ML 모델의 성공에 있어 양질의 데이터의 중요성과 MLOps의 핵심 개념, 파이프라인, 역할을 이해하고 데이터 중심 AI 접근 방식의 중요성을 강조합니다.

Contents

1	개요	2
2	용어 정리	3
3	강의 공지 및 과제	4
4	지난 강의 복습	5
5	핵심 개념: MLOps와 데이터의 중요성	6
5.1	MLOps 개요	6
5.2	ML 프로젝트의 주요 역할 (페르소나)	6
5.3	ML 워크플로우	7
5.4	데이터의 중요성: '빅 굿 데이터'의 필요성	8
5.4.1	좋은 데이터의 정의	8
5.4.2	AI 개선 전략: 모델 중심 vs. 데이터 중심 AI	8
5.5	MLOps 성숙도 모델	9
6	MLOps 파이프라인 상세	10
6.1	모델 훈련 파이프라인 (Model Training Pipeline)	10
6.2	모델 추론 파이프라인 (Model Inferencing Pipeline)	12
6.3	모델 모니터링 파이프라인 (Model Monitoring Pipeline)	12
6.4	데이터 드리프트 vs. 모델 드리프트	13
7	ML 모델 배포 전략	14
7.1	환경 및 배포 방식	14
7.2	MLFlow를 활용한 워크플로우	14
7.3	하이퍼파라미터 튜닝 (Hyperparameter Tuning)	15
7.4	자동화: 웹훅 (Webhooks)	15
7.5	CI/CD 전체 그림 (Full CI/CD Picture)	16
8	실습: 고객 이탈 예측 모델	17
8.1	실습 개요	17
8.2	데이터 준비 및 특성 공학 (Data Prep & Feature Engineering)	18
8.3	모델 훈련 (Model Training)	18
8.4	모델 등록 및 버전 관리 (Model Registration & Versioning)	19
8.5	챌린저 모델 검증 (Challenger Model Validation)	20
8.6	배치 추론 (Batch Inference)	21
9	체크리스트	22

10 FAQ (자주 묻는 질문)	23
11 빠르게 훑어보기 (1 페이지 요약)	24

1 개요

□ 핵심 요약

본 문서는 'CSCI103 Lecture 9' 강의 내용을 바탕으로, 머신러닝(ML) 모델의 성공에 있어 **양질의 데이터가 얼마나 중요한지**를 심층적으로 다룹니다. 또한, ML 모델을 개발하고 배포하며 지속적으로 관리하는 일련의 과정인 **MLOps의 핵심 개념, 파이프라인, 그리고 다양한 역할**을 설명합니다. 비즈니스 목표를 ML 문제로 전환하는 과정부터 데이터 준비, 모델 훈련, 검증, 배포, 모니터링에 이르는 전 과정을 이해하고, 데이터 중심 AI 접근 방식의 중요성을 강조합니다.

2 용어 정리

다음 표는 본 문서에서 사용되는 주요 용어와 그 설명을 담고 있습니다.

Table 1: 주요 용어 정리

용어	쉬운 설명	원어	비고
MLOps	머신러닝 모델의 개발부터 운영까지 전 과정을 자동화하고 관리하는 방법론	Machine Learning Operations	DevOps, DataOps의 ML 버전
데이터 드리프트	모델 입력 데이터의 통계적 특성이 시간이 지남에 따라 변하는 현상	Data Drift	모델 성능 저하의 주요 원인
모델 드리프트	모델의 예측 성능이 시간이 지남에 따라 저하되는 현상	Model Drift	데이터 드리프트나 개념 드리프트로 인해 발생
개념 드리프트	입력 변수와 타겟 변수 간의 관계가 시간이 지남에 따라 변하는 현상	Concept Drift	비즈니스 환경 변화 등으로 발생
피처 엔지니어링	원시 데이터를 ML 모델에 적합한 형태로 가공하여 새로운 특성(피처)을 만드는 과정	Feature Engineering	모델 성능 향상에 결정적인 역할
피처 스토어	ML 모델에서 사용되는 특성(피처)들을 중앙에서 저장하고 관리하는 저장소	Feature Store	피처 재사용성 및 일관성 확보
오토ML	머신러닝 모델 개발 과정을 자동화하는 기술	AutoML	비전문가도 ML 모델을 쉽게 구축 가능
MLFlow	머신러닝 수명 주기 관리를 위한 오픈소스 플랫폼	MLFlow	실험 추적, 모델 관리, 모델 배포 기능 제공
MLFlow 레지스트리	MLFlow에서 모델 버전을 관리하고 배포 단계를 추적하는 중앙 저장소	MLFlow Registry	모델의 '진실의 원천(Source of Truth)' 역할
CI/CD	소프트웨어 개발 단계를 자동화하여 더 빠르고 안정적인 배포를 가능하게 하는 방법론	Continuous Integration / Continuous Deployment	MLOps의 핵심 요소
하이퍼파라미터 튜닝	ML 모델의 성능을 최적화하기 위해 하이퍼파라미터의 최적 조합을 찾는 과정	Hyperparameter Tuning	모델의 정확도와 일반화 능력에 영향
데이터 중심 AI	모델 코드보다는 데이터의 품질을 체계적으로 개선하여 AI 성능을 향상시키는 접근 방식	Data-centric AI	모델 성능 향상에 더 큰 효과를 줄 수 있음
모델 중심 AI	데이터는 고정하고 모델 코드나 알고리즘을 개선하여 AI 성능을 향상시키는 접근 방식	Model-centric AI	데이터 중심 AI와 상호 보완적

3 강의 공지 및 과제

이번 강의에서는 몇 가지 중요한 공지사항과 과제 관련 정보가 공유되었습니다.

- **사례 연구 1 (Case Study 1):** 현재 진행 중입니다.
- **과제 4 (Assignment 4):** 12일 내로 공개될 예정이며, 제출 기한은 약 3주입니다. 목요일 섹션에서 검토가 있을 예정입니다.
- **사례 연구 2 (Case Study 2):** 그룹 프로젝트로 진행되며, 추후 공개될 예정입니다.
- **최종 프로젝트 (Final Project):** 역시 그룹 프로젝트이며, 추후 진행됩니다. 이번 강의에서 다루는 ML 파이프라인 생성에 관련된 다양한 역할들이 그룹 프로젝트에 중요하게 작용할 것입니다.
- **퀴즈 1 (Quiz 1):** 이미 완료되었으며, 채점 결과는 제출 마감일로부터 2주 이내(다음 주 말까지)에 공개될 예정입니다.
- **퀴즈 2 (Quiz 2):** 11월 말에 공개될 예정입니다.
- **그룹 구성:** 사례 연구 2와 최종 프로젝트는 새로운 그룹으로 구성될 예정입니다. 이는 학생들이 더 많은 동료들과 교류하며 네트워크를 확장하고 새로운 관계를 형성할 수 있는 좋은 기회가 될 것입니다.

4 지난 강의 복습

지난 강의에서 다루었던 주요 개념들을 질문과 답변 형식으로 복습합니다.

1. 시간이 지남에 따라 모델이 오래되는 현상을 무엇이라고 부르나요?
 - **답변:** 드리프트 (Drift) 또는 모델 드리프트 (Model Drift), 데이터 드리프트 (Data Drift) 라고 합니다. 모델의 데이터가 변하거나, 모델의 맥락(비즈니스 의미론)이 변하여 모델의 효과가 떨어지는 현상을 말합니다.
2. MLFlow는 무엇인가요?
 - **답변:** 모델 수명 주기 관리(Model Life Cycle Management)를 위한 오픈소스 프레임워크입니다.
3. MLFlow 트래킹 서버(Tracking Server)는 무엇을 추적하나요?
 - **답변:** 실험의 모든 측면을 추적합니다. 예를 들어, 파라미터(parameters), 메트릭(metrics), 아티팩트(artifacts), 코드(code), 데이터(data), 태그(tags) 등 모델 실험의 모든 세부 정보를 기록하여 시간 경과에 따른 메트릭을 확인할 수 있도록 돕습니다.
4. MLFlow 레지스트리(Registry)는 무엇을 돕나요?
 - **답변:** 모델 관리를 돕는 일종의 카탈로그(catalog)입니다. 모든 모델 버전(versions)을 추적하고, 사람들이 모델을 쉽게 찾을 수 있도록 하며, 모델을 한 스테이징 영역(staging area)에서 다른 영역으로 승격(promote)시키는 데 도움을 줍니다.
5. MLFlow는 온프레미스(on-premise) 환경에서도 작동할 수 있나요?
 - **답변:** 네, 그렇습니다. 클라우드 환경뿐만 아니라 온프레미스 환경에서도 작동합니다.
6. 모델 입력에 데이터를 준비하는 과정을 무엇이라고 부르나요?
 - **답변:** 전처리(Preprocessing) 또는 피처 엔지니어링 (Feature Engineering) 이라고 합니다. 데이터 과학자는 모델에 정보를 입력하기 위해 피처(features)를 생성합니다. 예를 들어, 생년월일을 모델에 더 적합한 '나이'로 변환하는 것과 같습니다.
7. 모델 서빙(serving) 및 스코어링(scoring)의 유형은 무엇인가요?
 - **답변:** 배치(batch), 스트리밍(streaming), 실시간(real-time) 모델 서빙이 있습니다. 이는 데이터 파이프라인 논의와 유사합니다.
8. 오버피팅(Overfit)은 무엇을 의미하나요?
 - **답변:** 모델이 필요 이상으로 많은 속성(attributes)을 가지고 있거나, 특정 데이터 세트에 너무 잘 맞게 훈련되어 다른 입력 데이터 세트에서는 잘 작동하지 않는 현상을 말합니다. 과적합(overtrained) 되었다고 표현합니다.
9. 피처(Feature) 재사용을 돕는 것은 무엇인가요?
 - **답변:** 피처 스토어(Feature Store)입니다. 이는 피처 카탈로그와 유사하지만, 실제 피처와 계산된 피처를 저장합니다.
10. AutoML은 무엇을 의미하나요?
 - **답변:** 데이터를 전달하면 AI가 모델을 생성하는 AI 지원 모델 생성(AI assisted model creation)을 의미합니다. 이는 '시민 데이터 과학자(citizen data scientists)'가 모델을 더 쉽게 만들 수 있도록 돕지만, 일반적으로 모델 생성 과정에 대한 제어권이 줄어듭니다.

5 핵심 개념: MLOps와 데이터의 중요성

이번 강의의 핵심은 정확한 머신러닝 모델을 생성하는 데 있어 데이터의 중요성을 이해하는 것입니다. 이는 단순히 데이터 자체뿐만 아니라, 데이터 처리 과정(process)과 관련 인력(people)의 역할까지 포함합니다.

5.1 MLOps 개요

□ 핵심 요약

MLOps는 머신러닝(ML), 데이터 엔지니어링(Data Engineering), 데브옵스(DevOps)의 교차점에 있는 방법론입니다. ML 모델의 설계, 훈련, 배포, 모니터링 및 재훈련 과정을 자동화하고 표준화하여, 모델이 프로덕션 환경에서 안정적으로 작동하고 지속적으로 개선될 수 있도록 합니다.

MLOps는 모델이 개발 환경에서 프로덕션 환경으로 원활하게 이동하고, 시간이 지나도 성능을 유지하며, 필요에 따라 재훈련될 수 있도록 하는 전체 프로세스를 의미합니다. 이는 소프트웨어 엔지니어링, 데이터 엔지니어링, 데브옵스, 그리고 모델 훈련의 여러 측면을 통합합니다.

- **소프트웨어 엔지니어링 측면:** 모듈화된 코드 작성, 버전 관리(Git), CI/CD 파이프라인 구축.
- **데이터 엔지니어링 측면:** 데이터 파이프라인 구축, 데이터 품질 관리, 피처 스토어(Feature Store) 활용.
- **데브옵스 측면:** 지속적인 통합(CI) 및 지속적인 배포(CD)를 통한 자동화된 모델 배포.
- **모델 훈련 측면:** 모델 실험 추적, 하이퍼파라미터 튜닝, 모델 버전 관리.

5.2 ML 프로젝트의 주요 역할 (페르소나)

ML 모델을 성공적으로 구축하고 운영하기 위해서는 다양한 전문성을 가진 사람들이 협력해야 합니다. 각 역할은 ML 생태계에서 고유한 책임을 가집니다.

1. 비즈니스 이해관계자 (Business Stakeholders):

- **역할:** ML 솔루션을 통해 비즈니스 가치를 창출하는 데 관심이 있습니다. 비즈니스 문제를 정의하고, 모델의 목표와 기대 효과를 제시합니다.
- **예시:** 고객 이탈률 감소, 매출 증대, 비용 절감 등.

2. 데이터 엔지니어 (Data Engineer):

- **역할:** ML 모델에 필요한 데이터를 수집, 정리, 변환하여 데이터 파이프라인을 구축합니다. 데이터의 품질과 접근성을 보장합니다.
- **예시:** CRM 시스템, 웹 로그, 청구 데이터 등에서 데이터를 추출하여 정제된 형태로 제공.

3. 데이터 과학자 (Data Scientist):

- **역할:** 비즈니스 문제를 ML 문제로 번역하고, 데이터를 사용하여 피처를 계산하며, 모델을 구축하고 테스트합니다.
- **예시:** 고객 이탈 예측 모델 개발, 최적의 알고리즘 선택, 모델 성능 평가.

4. ML 엔지니어 (ML Engineer):

- **역할:** 데이터 과학자가 만든 모델을 프로덕션 환경에 배포하고, 재현 가능한 방식으로 운영하며, 시간이 지남에 따라 재훈련될 수 있도록 자동화합니다.
- **예시:** 모델을 API로 노출, 모니터링 대시보드 구축, 자동 재훈련 워크플로우 설정.

5. 데이터 거버넌스 담당자 (Data Governance Officer):

- **역할:** 데이터에 대한 적절한 보안 제어가 이루어지고, 데이터가 적절히 식명화되며, 규정 준수 (compliance)가 보장되도록 합니다.
- **예시:** 개인 정보 보호 규정(GDPR, HIPAA) 준수, 데이터 접근 권한 관리.

5.3 ML 워크플로우

ML 워크플로우는 데이터 준비부터 모델 모니터링까지 일련의 단계를 포함하며, 각 단계에서 다양한 역할들이 협력합니다.

Figure 1: ML 워크플로우 및 역할별 책임 (개념도)

단계	데이터 준비 (Data Prep)	피처 엔지니어링 (Feature Engineering)	모델 훈련 (Model Training)	모델 검증 (Model Validation)	배포 및 모니터링 (Deployment & Monitoring)
데이터 거버넌스	전 과정에 걸쳐 데이터 규정 준수 및 보안 보장				
데이터 엔지니어	데이터 수집, 정제, 통합, 구조화				
데이터 과학자	데이터 탐색 및 분석	피처 생성, 변환	모델 선택, 훈련	모델 성능 평가	
ML 엔지니어					모델 배포, 인프라 관리, 성능 모니터링, 재훈련
비즈니스 분석가/이해관계자				모델 결과 검토, 비즈니스 가치 평가	모델 활용, 비즈니스 영향 분석

- **데이터 준비 (Data Prep):** 데이터 엔지니어가 여러 소스에서 데이터를 가져와 정제하고 표준화하여 데이터 과학자가 사용할 수 있도록 준비합니다.
- **데이터 탐색 및 분석 (Data Exploration & Analysis):** 데이터 과학자가 데이터를 이해하고 패턴을 찾습니다.
- **피처 엔지니어링 (Feature Engineering):** 데이터 과학자가 원시 데이터를 모델에 유의미한 피처로 변환합니다.
- **모델 훈련 (Model Training):** 데이터 과학자가 다양한 알고리즘을 실험하고 모델을 훈련합니다.
- **모델 검증 (Model Validation):** 데이터 과학자와 비즈니스 이해관계자가 모델의 성능을 평가하고 비즈니스 목표에 부합하는지 확인합니다.
- **모델 배포 (Model Deployment):** ML 엔지니어가 검증된 모델을 프로덕션 환경에 배포하여 실제 예측에 사용될 수 있도록 합니다.
- **모델 모니터링 (Model Monitoring):** ML 엔지니어가 배포된 모델의 성능을 지속적으로 추적하고, 데이터 드리프트나 모델 성능 저하를 감지합니다.

5.4 데이터의 중요성: '빅 굿 데이터'의 필요성

□ 핵심 요약

AI의 성능을 향상시키는 데 있어 데이터의 품질은 모델 코드의 개선만큼, 혹은 그 이상으로 중요합니다. 단순히 많은 양의 데이터('빅 데이터')를 넘어, 고품질의 데이터('빅 굿 데이터')를 확보하는 것이 AI 성공의 핵심입니다.

우리는 방대한 양의 데이터('빅 데이터')를 가지고 있지만, 품질이 낮은 데이터는 여전히 모델에 제대로 기여하지 못합니다. 따라서 '빅 굿 데이터'를 확보하는 것이 중요합니다.

5.4.1 좋은 데이터의 정의

좋은 데이터는 다음과 같은 특성을 가집니다.

- **명확한 레이블 (Unambiguous Labels):** 지도 학습(supervised learning)의 경우, 모델이 학습할 '정답'인 레이블이 명확해야 합니다.
- **중요한 사용 사례 포괄 (Covers Important Use Cases):** 데이터가 특정 사용 사례를 대표할 수 있도록 충분히 다양하고 포괄적이어야 합니다. 예를 들어, 모든 주(state)에 대한 연구를 매사추세츠(Massachusetts) 데이터만으로 하는 것은 대표성이 부족합니다.
- **생산 데이터로부터의 시기적절한 피드백 (Timely Feedback from Production Data):** 모델의 예측이 실제 결과와 얼마나 일치하는지 빠르게 확인할 수 있어야 합니다. 피드백이 늦어지면 모델이 오래되거나 비효율적인 상태로 오래 운영될 수 있습니다.
- **적절한 크기 (Sized Appropriately):** 데이터의 양이 충분해야 합니다. 각 주에서 하나의 데이터 포인트만으로는 충분하지 않을 수 있습니다.

데이터는 AI의 '음식'입니다. AI는 데이터 포인트를 알고리즘에 공급하여 학습하고, 새로운 데이터 포인트가 들어왔을 때 무언가를 예측합니다. 따라서 음식의 품질이 중요하듯이, 데이터의 품질이 AI의 성능에 직접적인 영향을 미칩니다.

5.4.2 AI 개선 전략: 모델 중심 vs. 데이터 중심 AI

AI 성능을 개선하는 두 가지 주요 전략이 있습니다.

1. 모델 중심 AI (Model-centric AI):

- **목표:** 모델이나 코드를 변경하여 성능을 개선하는 방법입니다.
- **예시:** 새로운 알고리즘을 시도하거나, 기존 모델의 아키텍처를 최적화하거나, 하이퍼파라미터를 미세 조정하는 것.

2. 데이터 중심 AI (Data-centric AI):

- **목표:** 데이터를 체계적으로 변경하여 성능을 개선하는 방법입니다.
- **예시:** 데이터 정제, 누락된 값 처리, 이상치 제거, 새로운 피쳐 생성, 데이터 증강(data augmentation) 등.

주의사항

어떤 전략이 더 효과적일까요? 두 가지 접근 방식 모두 필요하지만, 일반적으로 **데이터 중심 AI**가 더 큰 성능 향상을 가져옵니다. 모델 코드를 아무리 정교하게 다듬어도, 입력 데이터의 품질이 낮으면 모델의 잠재력을 최대한 발휘하기 어렵기 때문입니다.

□ 예제

실제 사례 비교: 강의에서 제시된 실제 사례에서는 강철, 태양광 패널, 표면 검사에서 결함을 감지하는 모델의 성능을 개선하는 실험이 있었습니다.

- **모델 중심 접근:** 기준 성능(76% 85%)에서 0% 0.4%의 미미한 개선을 보였습니다.
- **데이터 중심 접근:** 기준 성능에서 3% 16%의 훨씬 더 큰 개선을 달성했습니다.

이는 데이터 품질 개선이 모델 코드 개선보다 훨씬 더 큰 '리프트(lift)'를 제공할 수 있음을 보여줍니다.

요리 비유: 고품질 재료(데이터)를 조달하고 준비하는 것이 요리(모델 훈련) 자체보다 더 중요합니다. 아무리 훌륭한 요리사라도 나쁜 재료로는 좋은 음식을 만들 수 없듯이, 아무리 좋은 알고리즘이라도 나쁜 데이터로는 좋은 모델을 만들 수 없습니다.

5.5 MLOps 성숙도 모델

MLOps의 성숙도는 조직이 ML 모델을 얼마나 효율적이고 안정적으로 개발, 배포, 운영하는지를 나타냅니다. 성숙도는 여러 핵심 역량을 통해 점진적으로 향상됩니다.

Figure 2: MLOps 성숙도 모델 (개념도)

레벨	특징	주요 역량
레벨 1: 초급 (Beginner)	익숙한 도구 사용, 생산성 향상, 복잡한 워크로드 계획	사람: 지속적인 교육 및 훈련 데이터: 기본적인 데이터 아키텍처 모델: 기본적인 모델 아키텍처 프로세스: 기본적인 품질 검사 및 승격 프로세스 거버넌스: 기본적인 데이터 접근 제어
레벨 2: 중급 (Intermediate)	데이터 및 워크로드 확장, 작업 자동화, 데이터 팀 통합	확장성: 대규모 데이터 및 워크로드 처리 (예: 멀티 GPU 훈련) 자동화: 수동 작업 최소화, 재현성 확보 협업: 이질적인 기술 스택을 사용하는 팀 간 통합 및 자산 재사용
레벨 3: 고급 (Advanced)	매우 빠른 프로덕션 주기, 엔드투엔드 자동화 및 재현성	속도: 하루에도 여러 번 배포 가능한 빠른 프로덕션 주기 (예: Facebook, Netflix) 자동화: 잘 정의된 프로세스와 엔드투엔드 자동화 복구력: 시스템 장애에 대한 빠른 복구 능력 (예: Chaos Monkey) 재해 복구 (DR): 멀티 클라우드 전략, 지역 간 DR 전략

- **Chaos Monkey (카오스 몽키):** Netflix에서 개발한 도구로, 의도적으로 프로덕션 시스템에 문제를 주입하여 시스템이 얼마나 빨리 복구되는지 테스트합니다. 이는 시스템의 복원력을 높이는 데 사용됩니다.
- **롤백 메커니즘 (Rollback Mechanism):** 배포된 변경 사항이 문제를 일으킬 경우, 이전 상태로 빠르게 되돌릴 수 있는 기능입니다. 이는 프로덕션 환경의 안정성을 유지하는 데 필수적입니다.
- **재해 복구 (Disaster Recovery):** 단일 클라우드 영역 또는 전체 지역의 장애 발생 시, 비즈니스 연속성을 보장하기 위한 전략입니다. 멀티 클라우드 전략이나 다른 지역으로의 폴백(fallback)이 포함될 수 있습니다.

6 MLOps 파이프라인 상세

MLOps는 모델 훈련, 추론, 모니터링의 세 가지 핵심 파이프라인으로 구성됩니다. 이들은 모델의 전체 수명 주기를 관리하며, 각 파이프라인은 특정 목적을 가집니다.

6.1 모델 훈련 파이프라인 (Model Training Pipeline)

□ 핵심 요약

모델 훈련 파이프라인은 원시 데이터를 가공하여 모델을 학습시키고, 최적의 성능을 내는 모델을 선정하여 배포 준비를 마치는 과정입니다. 목표는 훈련 데이터에 과적합되지 않으면서 유용한 패턴을 포착하는 잘 일반화된(well-generalized) 모델을 만드는 것입니다.

1. 데이터 준비 (Data Prep):

- **설명:** 데이터를 훈련(training), 검증(validation), 테스트(test) 세트로 분할합니다. 이는 모델의

Figure 3: MLOps의 3가지 핵심 파이프라인 (개념도)

훈련 파이프라인 (Training Pipeline)	추론 파이프라인 (Inferencing Pipeline)	모니터링 파이프라인 (Monitoring Pipeline)
- 원시 데이터 수집 및 준비 - 피처 엔지니어링 및 피처 스토어 저장 - 모델 훈련 및 평가 - 최적 모델 선택 및 레지스트리 등록 - 모델 배포 준비	- 새로운 고객 데이터 입력 - 배포된 모델을 사용하여 예측 수행 - 고객 스코어링 (예: 이탈 가능성) - 비즈니스 의사 결정 지원	- 데이터 드리프트 감지 - 모델 드리프트 감지 - 개념 드리프트 감지 - 모델 예측 품질 확인 (예측 vs 실제) - 이상 감지 시 재훈련 트리거 및 알림

- 이 세 파이프라인은 상호 연결되어 모델이 지속적으로 학습하고, 정확한 예측을 제공하며, 프로덕션 환경에서 안정적으로 유지되도록 합니다.
- 모니터링 파이프라인에서 문제가 감지되면, 훈련 파이프라인이 자동으로 재시작되어 모델을 업데이트합니다.

성능을 객관적으로 평가하고 과적합을 방지하는 데 필수적입니다.

- 예시:** 고객 이탈 예측 모델의 경우, 전체 고객 데이터를 훈련, 검증, 테스트 세트로 나누어 사용합니다.

2. 피처 엔지니어링 (Feature Engineering):

- 설명:** 원시 변수(raw variables)를 모델에 의미 있는 입력으로 변환합니다. 이는 모델의 예측력을 크게 향상시킬 수 있습니다.
- 예시:** 고객의 나이, 마지막 구매일, 통화 빈도 등을 이탈 예측에 유용한 피처로 변환합니다.

3. 모델 선택 및 훈련 (Model Selection & Training):

- 설명:** 특정 머신러닝 문제에 적합한 모델(알고리즘)을 선택하고 데이터를 사용하여 모델을 학습시킵니다.
- 예시:** 로지스틱 회귀(Logistic Regression), 랜덤 포레스트(Random Forest), XGBoost 등 다양한 알고리즘을 실험합니다.

4. 파라미터 튜닝 (Parameter Tuning):

- 설명:** 모델의 정확도를 개선하고 과적합을 줄이기 위해 하이퍼파라미터(hyperparameters)를 최적화합니다.
- 예시:** 신경망의 학습률(learning rate), 랜덤 포레스트의 트리의 개수(number of trees) 등을 조정합니다.

5. 평가 (Evaluation):

- 설명:** 모델의 성능을 측정하기 위해 다양한 메트릭을 사용합니다.
- 예시:** 분류 모델의 경우 정확도(accuracy), 정밀도(precision), 재현율(recall), F1 점수(F1 score)를 사용하고, 회귀 모델의 경우 RMSE(Root Mean Squared Error), MAE(Mean Absolute Error), R-제곱(R-squared) 등을 사용합니다.

6. 모델 영속화 (Model Persistence):

- 설명:** 잘 훈련된 모델을 아티팩트(artifact)로 저장하여 재사용 및 배포 준비를 합니다. **예시:** 모델 파일을 직렬화(serialize)하여 저장하고, MLFlow 레지스트리에 등록합니다.

6.2 모델 추론 파이프라인 (Model Inferencing Pipeline)

□ 핵심 요약

모델 추론 파이프라인은 훈련된 모델을 사용하여 새로운 데이터에 대한 예측 또는 결정을 생성하는 과정입니다. 이 예측은 비즈니스 의사 결정을 안내하는 데 사용됩니다.

1. 예측 생성 (Generate Predictions):

- **설명:** 모델을 실제 비즈니스 의사 결정에 활용하는 단계입니다.
- **예시:** 고객 이탈 예측 모델을 사용하여 다음 주에 이탈할 가능성이 있는 고객을 예측하고, 마케팅 팀이 이 고객들에게 연락하도록 합니다.

2. 추론 유형 (Inferencing Types):

- **배치 추론 (Batch Inferencing):** 대규모 데이터 세트에 대해 한 번에 예측을 수행합니다.
- **실시간 추론 (Real-time Inferencing):** 웹사이트 상호작용과 같이 즉각적인 예측이 필요한 경우에 사용됩니다. 고객 행동을 기반으로 실시간으로 이탈 가능성을 감지하고 할인 등의 제안을 할 수 있습니다.

3. 인프라 요구사항 (Infrastructure Requirements):

- **설명:** 모델 서빙 프레임워크, 입출력 처리, 성능(지연 시간, 처리량, 스케일링, 캐싱), 보안 등이 고려되어야 합니다.
- **예시:** 클라우드 인프라(AWS, Azure) 또는 엣지 시스템(edge systems)에서 모델 서빙을 최적화합니다.

6.3 모델 모니터링 파이프라인 (Model Monitoring Pipeline)

□ 핵심 요약

모델 모니터링 파이프라인은 프로덕션 환경에서 배포된 모델의 동작과 성능을 지속적으로 추적하여 데이터 드리프트, 개념 드리프트, 모델 성능 저하와 같은 문제를 감지하는 과정입니다.

1. 문제 감지 (Detecting Issues):

- **데이터 드리프트 (Data Drift):** 입력 데이터 분포가 시간이 지남에 따라 변하는 현상입니다. 새로운 고객 인구 통계나 제품 카테고리 변화 등이 원인이 될 수 있습니다.
- **개념 드리프트 (Concept Drift):** 입력과 타겟 변수 간의 관계가 변하는 현상입니다. 새로운 가격 전략이나 서비스 비용 상승 등으로 고객 행동이 변하는 것이 예시입니다.
- **예측 품질 (Prediction Quality):** 모델의 예측이 실제 결과와 얼마나 일치하는지 비교합니다. 모델이 이탈하지 않을 것이라고 예측한 고객이 실제로 이탈하는 경우, 모델의 예측 품질이 저하되었음을 의미합니다.

2. 도구 및 알림 (Tools & Alerting):

- **설명:** MLOps 플랫폼과 대시보드를 사용하여 추세를 확인하고 성능 저하를 감지하며, 알림을 생성합니다.
- **예시:** 모델 성능이 특정 임계값을 초과하여 저하되면 데이터 엔지니어와 데이터 과학자에게 자동으로 알림이 전송됩니다.

3. 자동 재훈련 (Automated Retraining):

- **설명:** 모델 드리프트가 특정 임계값을 초과하는 등 문제가 감지되면, 모델을 새로운 데이터로

자동으로 재훈련하도록 설정할 수 있습니다. **예시:** 데이터 드리프트가 감지되면, 최신 데이터를 사용하여 모델을 재훈련하는 작업이 자동으로 시작됩니다.

6.4 데이터 드리프트 vs. 모델 드리프트

이 두 가지 개념은 모델 성능 저하와 관련이 있지만, 원인과 감지 방식에서 차이가 있습니다.

Table 2: 데이터 드리프트와 모델 드리프트 비교

특징	데이터 드리프트 (Data Drift)	모델 드리프트 (Model Drift)
정의	모델 입력 데이터의 통계적 특성 변화	모델의 예측 성능 저하
원인	데이터 스키마 변경, 데이터 분포 변화, 새로운 고객 인구 통계, 새로운 제품 카테고리 등	데이터 드리프트, 개념 드리프트, 비즈니스 맥락 변화 (예: COVID-19 팬데믹)
감지 방법	입력 데이터의 스키마, 분포 변화 모니터링	모델의 정확도, 정밀도, 재현율 등 성능 지표 모니터링
결과	모델이 학습한 패턴이 더 이상 유효하지 않게 됨	모델의 예측이 현실과 불일치하게 됨
해결책	새로운 데이터로 모델 재훈련	새로운 데이터로 모델 재훈련, 새로운 피처 추가, 모델 재설계

- 데이터 드리프트는 모델에 들어오는 데이터 자체의 변화를 의미합니다.
- 모델 드리프트는 모델의 예측 정확도가 떨어지는 현상을 의미하며, 데이터 드리프트나 비즈니스 환경 변화 등 다양한 원인으로 발생할 수 있습니다.

7 ML 모델 배포 전략

ML 모델을 개발 환경에서 프로덕션 환경으로 배포하는 과정은 여러 단계를 거치며, 효율성과 안정성을 위해 신중한 전략이 필요합니다.

7.1 환경 및 배포 방식

일반적으로 개발(Dev), 스테이징(Staging), 프로덕션(Prod)의 세 가지 환경이 존재합니다. 모델 배포에는 크게 두 가지 전략이 있습니다.

1. 모델 배포 (Deploy Model):

- **설명:** 개발 환경에서 모델을 한 번 구축하고, 이 모델 아티팩트(artifact)를 다른 환경으로 승격(promote)시키는 방식입니다.
- **장점:** 계산 비용이 적고, 배포 속도가 빠릅니다.
- **단점:** 각 환경의 데이터 특성이 다를 경우, 프로덕션 환경에서 모델 성능이 저하될 수 있습니다.

2. 코드 배포 (Deploy Code):

- **설명:** 모델을 구축하는 로직(코드)을 각 환경으로 배포하고, 해당 환경의 데이터를 사용하여 모델을 재훈련하는 방식입니다.
- **장점:** 각 환경의 실제 데이터를 사용하여 모델을 훈련하므로 더 정확하고, 데이터 드리프트에 강하며, 데브옵스 관점에서 더 깔끔합니다. 재현 가능한 모델을 만들고, 대규모 프로젝트를 지원하는 데 유리합니다. **단점:** 각 환경에서 재훈련이 필요하므로 계산 비용이 더 많이 들고, 데이터 과학 팀이 모듈화된 코드를 작성해야 합니다.

주의사항

실제 데이터의 중요성: 시뮬레이션된 데이터로 테스트할 때는 발견되지 않았던 문제가 실제 프로덕션 데이터의 불균형이나 패턴(데이터 스쿠) 때문에 발생할 수 있습니다. 따라서 '코드 배포' 전략을 통해 실제 프로덕션 데이터로 모델을 재훈련하는 것이 중요합니다.

7.2 MLFlow를 활용한 워크플로우

MLFlow는 ML 모델의 수명 주기 관리를 위한 강력한 도구이며, 다음 단계에서 활용됩니다.

1. 데이터 준비 및 피처화 (Data Prep & Featurization):

- **설명:** 노트북(notebooks), SQL 에디터 등을 사용하여 데이터를 준비하고 피처를 생성합니다. 데이터 과학자와 데이터 엔지니어가 협력합니다.

2. 모델 개발 (Model Development):

- **설명:** MLFlow 트래킹(Tracking) 기능을 사용하여 파라미터, 메트릭, 메타데이터, 모델 등 모든 실험 정보를 기록합니다. 다양한 프레임워크(예: scikit-learn, PyTorch, TensorFlow)로 모델을 구축할 수 있습니다.

3. 모델 레지스트리 (Model Registry):

- **설명:** 가장 성능이 좋은 모델은 MLFlow 레지스트리로 이동하여 테스트되고 버전 관리됩니다. Unity Catalog와 같은 중앙 엔티티를 통해 여러 워크스페이스(개발, 스테이징, 프로덕션)에서 모델에 접근하고, 접근 권한, 계보(lineage), 감사 정보 등을 관리할 수 있습니다.
- **MLFlow 레지스트리의 역할:** 모델의 '진실의 원천(Source of Truth)' 역할을 합니다. 어떤 모델

버전이 어디에 배포되었는지, 누가 만들었는지, 언제 승인되었는지 등 모든 메타데이터를 기록하여 추적성을 보장합니다.

4. 배포 (Deployment):

- **설명:** ML 엔지니어가 CI/CD 파이프라인을 관리하여 모델을 프로덕션 환경에 배포합니다. 개발에 사용된 프레임워크와 배포 프레임워크가 다를 수 있습니다.

코드 저장소 vs. 모델 저장소:

- **코드 저장소:** Git 과 같은 시스템을 사용하여 코드, 클러스터 구성, YAML 파일, pip 요구사항 등을 저장합니다.
- **모델 저장소:** MLFlow 레지스트리가 모델 아티팩트, 버전, 메타데이터를 저장하는 역할을 합니다. 이는 모델의 상태와 이력을 추적하는 데 필수적입니다.
- **데이터:** 데이터 자체는 클라우드 스토리지에 저장되며, Delta Lake와 같은 기술을 사용하여 버전 관리 및 계보 정보를 유지합니다. 모델 훈련에 사용된 특정 데이터 세트와 버전을 참조하는 것이 중요합니다.

7.3 하이퍼파라미터 튜닝 (Hyperparameter Tuning)

하이퍼파라미터 튜닝은 모델의 성능을 최적화하기 위해 하이퍼파라미터의 최적 조합을 찾는 과정입니다.

1. 수동 그리드 서치 (Manual Grid Search):

- **설명:** 모든 가능한 하이퍼파라미터 조합을 시도하는 방법입니다. K-겹 교차 검증(k-fold cross-validation)을 사용하여 훈련 및 검증 세트를 여러 번 분할하고 평균 오류를 계산하여 과소적합(underfit) 및 과적합(overfit)을 방지합니다.
- **단점:** 파라미터 수가 많아지면 계산 비용이 매우 비싸집니다.

2. 자동화된 서치 (Automated Search):

- **설명:** Optuna, Hyperopt와 같은 라이브러리를 사용하여 하이퍼파라미터 공간을 효율적으로 탐색합니다.
- **유형:**
 - **랜덤 서치 (Random Search):** 무작위로 하이퍼파라미터 조합을 선택합니다.
 - **베이저안 서치 (Bayesian Search):** 이전 실험 결과를 바탕으로 다음 시도할 하이퍼파라미터 조합을 지능적으로 선택하여 더 빠르게 최적점에 수렴합니다.
 - **유전 알고리즘 (Genetic Algorithms):** 좋은 조합과 나쁜 조합을 섞어 새로운 조합을 생성하는 방식으로, 특히 대규모 딥러닝 모델에서 효과적일 수 있습니다.

7.4 자동화: 웹훅 (Webhooks)

MLOps 파이프라인에서 모델의 환경 전환을 자동화하는 데 웹훅(webhooks)이 중요한 역할을 합니다.

- **설명:** MLFlow 레지스트리에서 모델을 스테이징 환경으로 전환하는 요청이 발생하면, 이 요청은 자동으로 웹훅을 트리거할 수 있습니다.
- **기능:**
 - **자동 테스트:** 웹훅은 Jobs API를 통해 자동화된 테스트 단계를 시작할 수 있습니다.
 - **알림:** Slack과 같은 메시징 서비스로 알림을 보내 새로운 모델이 레지스트리에 푸시되었음을 관련 팀에 알릴 수 있습니다.

- **태그 관리:** 모델이 스테이징 또는 프로덕션으로 푸시될 때 자동으로 태그가 지정되며, 사용이 중단되면 '아카이브(archived)' 태그가 지정될 수 있습니다.
- **MLFlow API:** MLFlow API는 이러한 웹훅과 연동하여 전체 알림 및 자동화 프로세스를 쉽게 만듭니다.

7.5 CI/CD 전체 그림 (Full CI/CD Picture)

다음은 Git 기반의 소스 제어, 피처 테이블 등을 활용한 CI/CD(지속적 통합/지속적 배포)의 전체 워크플로우를 보여주는 개념도입니다.

Figure 4: MLOps CI/CD 파이프라인 (개념도)

개발 환경 (Development)	스테이징 환경 (Staging) - CI	프로덕션 환경 (Production) - CD	모니터링 (Monitoring)
• Git 저장소 (Dev Branch): 코드 개발 • 데이터 (Delta Tables): EDA 및 훈련 데이터 • 피처 엔지니어링: 피처 생성 • 모델 훈련: 모델 반복 훈련 • 코드 커밋: Dev Branch에 코드 커밋	• Merge Request: Staging Branch로 병합 요청 • CI 트리거: 자동 단위 테스트 (Unit Test) • 통합 테스트 (Integration Test): 피처 테이블 및 데이터 테이블 사용 • Main Branch 병합: 테스트 통과 후 Main Branch로 병합	• Release Branch 생성: Main Branch에서 Release Branch 생성 • 모델 재훈련: 프로덕션 데이터로 모델 재훈련 • 모델 레지스트리 등록: 새로운 모델 버전 등록 • 모델 배포: 프로덕션 환경에 모델 배포	• 추론 및 서빙: 배포된 모델 사용 • 데이터 드리프트 감지: 입력 데이터 통계 모니터링 • 모델 드리프트 감지: 모델 성능 지표 모니터링 • 개념 드리프트 감지: 입력-타겟 관계 모니터링 • 알림 및 재훈련 트리거: 문제 감지 시 알림 및 자동 재훈련

- **녹색 화살표:** 데이터 읽기 (Reads)
- **빨간색 화살표:** 데이터 쓰기 (Writes)
- **뇌 모양 아이콘:** 모델 (Model)
- **주황색/노란색 아이콘:** 모델 전환 (Model Transition)

8 실습: 고객 이탈 예측 모델

이번 실습에서는 Databricks 환경에서 고객 이탈(Customer Churn) 예측 모델을 구축하는 엔드투엔드 MLOps 파이프라인을 살펴봅니다.

8.1 실습 개요

□ 핵심 요약

Databricks dbdemos 라이브러리를 사용하여 고객 이탈 예측 모델을 구축하는 MLOps 엔드투엔드 파이프라인을 실습합니다. 이 실습은 데이터 준비, 피처 엔지니어링, 모델 훈련, 모델 등록, 챌린저 모델 검증, 그리고 배치 추론의 전 과정을 포함합니다.

- **목표:** 마케팅 팀의 요청에 따라 고객 이탈 위험 점수(risk score)를 생성하는 모델을 개발합니다.
- **데이터셋:** 데이터 엔지니어 팀이 제공한 `MLOps_churn_bronze_customer`.
- **주요 도구:**
 - **데이터 관리:** Delta Lake (Unity Catalog를 통한 테이블 관리, 트랜잭션 보장, 타임 트래블)
 - **모델 관리:** MLFlow (실험 추적, 모델 레지스트리)
 - **모델 알고리즘:** LightGBM
- **워크플로우:**
 1. 데이터 준비 및 피처화 (Data Prep & Featurization)
 2. 모델 훈련 (Model Training)
 3. 모델 등록 (Putting Models into Unity Catalog)
 4. 챌린저 모델 검증 (Challenger Model Validation)
 5. 배치 추론 (Batch Inference)

□ 예제

Databricks 데모 설치: Databricks 노트북에서 다음 명령어를 실행하여 MLOps 엔드투엔드 파이프라인 데모를 설치할 수 있습니다.

```
1 %pip install dbdemos
2 import dbdemos
3 dbdemos.install("mlops-end-to-end-pipeline", catalog="CSCI103", schema="
  MLOps", create_schema=True)
```

Listing 1: Databricks MLOps 데모 설치

이 명령은 CSCI103 카탈로그 내에 MLOps 스키마를 생성하고, 관련 코드와 데이터를 배포합니다.

8.2 데이터 준비 및 특성 공학 (Data Prep & Feature Engineering)

□ 핵심 요약

고객 이탈 예측을 위한 원시 데이터를 탐색하고, 결측값 처리, 데이터 타입 변환, 새로운 피처 생성 등 모델 훈련에 적합한 형태로 가공하는 과정입니다.

1. 데이터 탐색 (Exploratory Data Analysis, EDA):

- 데이터셋 구성: $MLOps_{churn_{bronze}_{customer}}$ ID, , , , (tenure), , , , ,
- Pandas API on Spark: Spark 데이터프레임을 Pandas API를 사용하여 쉽게 조작할 수 있습니다. 이는 Pandas에 익숙한 데이터 과학자들에게 편리함을 제공합니다.

```
1 # Spark 을 DataFrame Pandas 로 API 변환
2 telco_df_pandas = telco_df.to_pandas_on_spark()
3 # 인터넷서비스제공자별카운트확인
4 telco_df_pandas['InternetService'].value_counts()
```

Listing 2: Pandas API on Spark 예시

2. 데이터 정제 및 피처화 (Data Cleansing & Featurization):

- 목표: 모델의 예측력을 높이기 위해 데이터를 정리하고 새로운 피처를 생성합니다.
- 예시 작업:
 - 선택적 서비스 수 계산: 고객이 가입한 선택적 서비스(온라인 보안, 온라인 백업 등)의 수를 계산하여 새로운 피처로 추가합니다. 서비스 수가 많을수록 이탈 가능성이 낮을 수 있습니다.
 - 의미 있는 레이블 제공: 'SeniorCitizen'과 같은 컬럼의 0/1 값을 'Yes'/'No'와 같은 의미 있는 문자열로 변환합니다.
 - 결측값 처리: 'TotalCharges'와 같은 컬럼의 결측값을 적절한 값(예: 0)으로 대체합니다.
 - 데이터 타입 변환: 필요한 경우 컬럼의 데이터 타입을 변경합니다.
- 결과 저장: 정제되고 피처화된 데이터는 $MLOps_{churn_{training}}$

8.3 모델 훈련 (Model Training)

□ 핵심 요약

피처 엔지니어링을 거친 데이터를 사용하여 LightGBM 모델을 훈련하고, MLFlow 트래킹을 통해 모든 실험 과정과 모델 아티팩트를 기록합니다.

1. MLFlow 실험 추적 (MLFlow Experiment Tracking):

- 실험 설정: MLFlow를 사용하여 실험 이름을 설정하고, 고유한 실험 ID를 얻습니다.
- 데이터 계보(Lineage) 기록의 중요성: 모델 훈련에 사용된 데이터셋의 정보를 MLFlow에 기록하는 것은 매우 중요합니다. 이는 나중에 모델의 예측 결과에 문제가 생겼을 때, 어떤 데이터로 모델이 훈련되었는지 추적(root cause analysis)하는 데 필수적입니다.

```
1 import mlflow
2 # 훈련데이터셋의최신버전을 MLFlow 입력으로기록
3 mlflow.log_input(mlflow.data.load_delta(f"MLOps.
    MLOps_churn_training@latest"), "training_input")
```

Listing 3: MLFlow를 이용한 데이터셋 계보 기록

- **메타데이터 기록:** MLFlow는 파라미터, 메트릭, 그래프, 모델 아티팩트뿐만 아니라, 모델 훈련에 사용된 데이터셋의 메타데이터(테이블 이름, 버전 등)도 함께 기록합니다.
2. 데이터 전처리 파이프라인 구축 (Data Preprocessing Pipeline):
- **다양한 데이터 타입 처리:** 불리언(boolean), 수치형(numerical), 범주형(categorical) 데이터는 각각 다른 전처리 방식이 필요합니다.
 - **Scikit-learn 파이프라인:** ColumnTransformer와 OneHotEncoder, StandardScaler 등을 사용하여 각 데이터 타입에 맞는 전처리 단계를 정의하고, 이를 하나의 전처리 파이프라인으로 통합합니다.
3. 모델 훈련 함수 정의 (Define Model Training Function):
- **LightGBM 분류기:** LGBMClassifier를 사용하여 고객 이탈 예측 모델을 구축합니다.
 - **자동 로깅 (Autolog):** mlflow.lightgbm.autolog()를 사용하여 모델의 모든 특성(파라미터, 메트릭, 아티팩트 등)을 자동으로 기록합니다.
 - **모델 서명 (Model Signature):** 모델의 입력 및 출력 스키마를 추론하여 기록합니다.
 - **평가:** 훈련 세트와 검증 세트에 대해 모델을 평가하고, 손실(loss) 및 메트릭(metrics)을 기록합니다.
4. 모델 훈련 실행 및 결과 확인 (Execute Model Training Review Results):
- **하이퍼파라미터 설정:** 람다(lambda), 학습률(learning rate), 최대 빈(max bins) 등 하이퍼파라미터를 설정하고 훈련 함수를 호출합니다.
 - **MLFlow UI:** MLFlow UI에서 단일 실험 실행(run)의 상세 정보를 확인할 수 있습니다. 여기에는 모델의 파이프라인 구조(전처리 단계, 분류기), 훈련 메트릭(정밀도-재현율 곡선, ROC 곡선, 혼동 행렬), 모델 아티팩트(requirements.txt, YAML 파일, pickle 형식의 모델 파일, 입력 예시 JSON 등)가 포함됩니다.
 - **아티팩트 다운로드:** MLFlow API를 사용하여 훈련된 모델의 아티팩트를 다운로드하고, 혼동 행렬(confusion matrix)과 같은 시각화 자료를 확인할 수 있습니다.

8.4 모델 등록 및 버전 관리 (Model Registration & Versioning)

□ 핵심 요약

훈련된 모델 중 최적의 모델을 MLFlow 레지스트리에 등록하고, 버전 관리, 설명 추가, 태그 지정을 통해 모델의 생명 주기를 체계적으로 관리합니다.

1. 최적 모델 검색 및 등록 (Search Register Best Model):

- **최적 실행(Run) 찾기:** MLFlow API를 사용하여 특정 실험 내에서 F1 점수(F1 score)가 가장 높은 실행(run)을 검색하여 최적 모델을 식별합니다.
- **모델 등록:** 식별된 최적 모델을 `mlflow.register_model` *UnityCatalog MLFlow* . 1 .

• 모델 메타데이터 관리 (Manage Model Metadata):

- **모델 설명 추가:** `MLFlowClient().update_registered_model` *UnityCatalog* .
- **버전별 설명 추가:** `MLFlowClient().update_model_version` .
- **태그 설정:** `MLFlowClient().set_model_version_tag` (: F1) .

• 모델 계보 및 추적성 확인 (Verify Model Lineage Traceability):

- **MLFlow UI에서 확인:** MLFlow 레지스트리에서 등록된 모델의 상세 페이지를 확인하면, 모델 훈련에 사용된 데이터셋(테이블 이름, 버전)과 해당 모델을 구축하는 데 사용된 노트북(코드)의 링크를 확인할 수 있습니다.
- **추적성(Traceability):** 이는 모델의 예측 결과에 대한 문제가 발생했을 때, 어떤 데이터와 코드로 모델이 만들어졌는지 정확히 추적할 수 있게 해주는 핵심 기능입니다.
- **챌린저 모델 설정 (Set Challenger Model):**
 - **별칭(Alias) 사용:** 새로 생성된 모델 버전은 'challenger'라는 별칭으로 설정됩니다. 이는 나중에 프로덕션에 있는 'champion' 모델과 비교하는 데 사용됩니다.

8.5 챌린저 모델 검증 (Challenger Model Validation)

□ 핵심 요약

새로 등록된 '챌린저' 모델이 기존 '챔피언' 모델보다 성능이 우수한지 검증하고, 비즈니스 가치 측면에서 평가하여 프로덕션 배포 여부를 결정합니다.

- 챌린저 모델 로드 (Load Challenger Model):**
 - **별칭으로 모델 가져오기:** MLFlow 클라이언트를 사용하여 'challenger' 별칭이 지정된 모델 버전을 가져옵니다.
- 자동화된 품질 검사 (Automated Quality Checks):**
 - **메타데이터 검증:** 모델에 설명이 있는지, F1 점수가 특정 임계값보다 높은지 등 정의된 품질 기준을 자동으로 확인합니다.
 - **합격/불합격 판단:** 모든 검사를 통과하면 'PASS'로 판단합니다.
- 챔피언 모델과의 비교 (Comparison with Champion Model):**
 - **F1 점수 비교:** 챌린저 모델의 F1 점수를 현재 프로덕션에 배포된 '챔피언' 모델의 F1 점수와 비교합니다.
 - **최초 배포:** 만약 챔피언 모델이 존재하지 않는 경우(최초 배포), 챌린저 모델은 자동으로 승인됩니다.
 - **승격 조건:** 챌린저 모델의 F1 점수가 챔피언 모델보다 높거나, 챔피언 모델이 없는 경우에만 챌린저 모델이 승격됩니다.
- 비즈니스 가치 평가 (Business Value Assessment):**
 - **이탈 고객 가치:** 이탈하는 고객 한 명당 비즈니스에 미치는 금전적 손실을 추정합니다.
 - **모델 가치 계산:** 챌린저 모델이 더 많은 이탈 고객을 정확하게 예측하고 이탈을 방지할 수 있다면, 그로 인한 비즈니스 가치(달러 단위)를 계산합니다. 이는 모델 배포의 경제적 타당성을 입증하는 데 사용됩니다.
- 챌린저 모델을 챔피언으로 승격 (Promote Challenger to Champion):**
 - **별칭 변경:** 검증을 통과하고 챔피언 모델보다 우수하다고 판단되면, 챌린저 모델의 별칭을 'champion'으로 변경하여 프로덕션 환경에 배포되었음을 나타냅니다.

```

1 # 챌린저모델의별칭을 'champion으로' 설정
2 client.set_registered_model_alias(
3     name=model_name,
4     alias="champion",
5     version=challenger_model_version.version
6 )

```

Listing 4: 챌린저 모델을 챔피언으로 승격

8.6 배치 추론 (Batch Inference)

□ 핵심 요약

프로덕션 환경에 배포된 '챔피언' 모델을 사용하여 새로운 고객 데이터에 대한 이탈 예측을 일괄적으로 수행하고, 그 결과를 확인합니다.

(a) 챔피언 모델 로드 (Load Champion Model):

- **별칭으로 모델 가져오기:** MLFlow 클라이언트를 사용하여 'champion' 별칭이 지정된 모델 버전을 가져옵니다.

(b) 새로운 데이터 준비 (Prepare New Data):

- **시뮬레이션:** 실제 시나리오를 시뮬레이션하기 위해 새로운 고객 데이터를 준비합니다. 이는 일반적으로 모델 훈련에 사용되지 않은 '홀드아웃(hold-out)' 데이터셋이거나, 실제 프로덕션에서 수집된 최신 데이터입니다.

(c) 모델을 이용한 예측 (Predict using the Model):

- **Spark UDF 활용:** Spark 사용자 정의 함수(UDF)를 사용하여 챔피언 모델을 새로운 데이터 프레임에 적용하고, 각 고객에 대한 이탈 예측 결과를 새로운 컬럼('prediction')으로 추가합니다.

```

1 import mlflow.pyfunc
2 # MLFlow pyfunc 모델로드
3 champion_pyfunc_model = mlflow.pyfunc.load_model(f"models:/{"
4     model_name}@champion")
5
6 # Spark 를 UDF 사용하여 예측 컬럼 추가
7 inference_df_with_predictions = spark.createDataFrame(
8     champion_pyfunc_model.predict(inference_data.to_pandas())
9 )
10 # 예측 결과가 포함된 데이터 프레임 표시
11 inference_df_with_predictions.display()

```

Listing 5: 챔피언 모델을 이용한 배치 추론

(d) 결과 확인 (Review Results):

- **예측 컬럼:** 최종 데이터 프레임에는 각 고객의 원래 정보와 함께 'prediction' 컬럼에 이탈 예측 결과('Yes' 또는 'No')가 포함됩니다. 이 결과는 비즈니스 팀이 이탈 가능성이 높은 고객에게 선제적으로 대응하는 데 사용됩니다.

9 체크리스트

이 체크리스트는 강의 내용을 학습하고, MLOps 프로젝트를 수행하며, 본 문서를 검토하는 데 도움이 됩니다.

- **MLOps 기본 이해:**

- ☐ MLOps의 정의와 필요성을 이해했는가?
- ☐ ML 프로젝트의 주요 5가지 역할(페르소나)을 설명할 수 있는가?
- ☐ ML 워크플로우의 주요 단계와 각 단계별 책임자를 파악했는가?

- **데이터의 중요성:**

- ☐ '빅 굿 데이터'의 의미와 좋은 데이터의 특성을 설명할 수 있는가?
- ☐ 데이터 중심 AI와 모델 중심 AI의 차이점과 각각의 장단점을 이해했는가?
- ☐ 데이터 중심 AI가 모델 성능 향상에 더 큰 영향을 미칠 수 있는 이유를 설명할 수 있는가?

- **MLOps 파이프라인:**

- ☐ 모델 훈련, 추론, 모니터링 파이프라인의 목적과 주요 단계를 설명할 수 있는가?
- ☐ 데이터 드리프트, 모델 드리프트, 개념 드리프트의 차이점과 감지 방법을 이해했는가?
- ☐ 드리프트 감지 시 자동 재훈련의 중요성을 설명할 수 있는가?

- **MLFlow 활용:**

- ☐ MLFlow 트래킹 서버가 무엇을 기록하는지 알고 있는가?
- ☐ MLFlow 레지스트리의 역할과 모델 버전 관리의 중요성을 이해했는가?
- ☐ MLFlow를 사용하여 모델의 데이터 계보(lineage)를 기록하는 방법을 이해했는가?

- **모델 배포 및 자동화:**

- ☐ '모델 배포'와 '코드 배포' 전략의 차이점과 각각의 장단점을 이해했는가?
- ☐ 하이퍼파라미터 튜닝의 필요성과 수동/자동화된 튜닝 방법(그리드, 베이지안, 유전)을 알고 있는가?
- ☐ 웹훅이 MLOps 파이프라인 자동화에 어떻게 기여하는지 설명할 수 있는가?
- ☐ CI/CD 파이프라인의 각 단계(개발, 스테이징, 프로덕션, 모니터링)를 설명할 수 있는가?

- **MLOps 성숙도:**

- ☐ MLOps 성숙도 모델의 각 레벨(초급, 중급, 고급)의 특징을 이해했는가?
- ☐ Chaos Monkey, 롤백 메커니즘, 재해 복구(DR)의 개념을 설명할 수 있는가?

- **실습 내용:**

- ☐ 고객 이탈 예측 모델 실습의 전체 워크플로우를 이해했는가?
- ☐ 데이터 준비, 피처 엔지니어링, 모델 훈련, 등록, 검증, 추론 각 단계에서 어떤 작업이 수행되는지 설명할 수 있는가?
- ☐ 챌린저 모델과 챔피언 모델의 개념을 이해하고, 모델 승격 과정을 설명할 수 있는가?

10 FAQ (자주 묻는 질문)

- **Q: 데이터 드리프트와 모델 드리프트는 항상 함께 발생하나요?**

- A: 아닙니다. 데이터 드리프트는 모델 드리프트의 흔한 원인이지만, 유일한 원인은 아닙니다. 비즈니스 맥락의 변화(개념 드리프트)나 외부 요인으로 인해 데이터 분포는 그대로인데 모델의 예측 성능만 저하될 수도 있습니다. 예를 들어, 팬데믹과 같은 예상치 못한 사건은 데이터 자체의 변화 없이도 모델의 유효성을 떨어뜨릴 수 있습니다.

- **Q: AutoML을 사용하면 데이터 과학자의 역할이 줄어드나요?**

- A: AutoML은 모델 개발의 일부 반복적인 작업을 자동화하여 데이터 과학자의 생산성을 높일 수 있습니다. 그러나 비즈니스 문제 정의, 데이터 이해 및 정제, 피처 엔지니어링, 모델 해석 및 비즈니스 영향 분석 등 고수준의 전문성은 여전히 데이터 과학자의 중요한 역할로 남아있습니다. AutoML은 '시민 데이터 과학자'에게 모델 구축을 용이하게 하지만, 모델에 대한 세밀한 제어는 어려울 수 있습니다.

- **Q: MLFlow 레지스트리가 모델의 '진실의 원천(Source of Truth)'이라는 것은 무슨 의미인가요?**

- A: 이는 MLFlow 레지스트리가 특정 모델의 모든 버전, 해당 모델을 누가 만들었는지, 어떤 파라미터와 데이터로 훈련되었는지, 어떤 메트릭을 달성했는지 등 모델에 대한 모든 공식적이고 신뢰할 수 있는 정보를 중앙에서 관리한다는 의미입니다. 이를 통해 모델의 이력을 완벽하게 추적하고, 재현성을 보장하며, 팀 간의 협업을 용이하게 합니다.

- **Q: 모델을 배포할 때 '모델 배포'와 '코드 배포' 중 어떤 전략을 선택해야 하나요?**

- A: 상황에 따라 다릅니다. '모델 배포'는 빠르고 간단하지만, 프로덕션 데이터의 특성이 개발 환경과 크게 다를 경우 성능 문제가 발생할 수 있습니다. '코드 배포'는 각 환경의 실제 데이터로 모델을 재훈련하므로 더 정확하고 안정적이지만, 계산 비용이 더 들고 CI/CD 파이프라인 구축이 필요합니다. 일반적으로 대규모의 중요한 프로덕션 시스템에서는 '코드 배포'가 더 선호됩니다.

- **Q: MLOps 성숙도가 낮은 조직이 가장 먼저 집중해야 할 부분은 무엇인가요?**

- A: 초급 단계의 조직은 먼저 익숙한 도구를 사용하여 생산성을 높이고, 기본적인 데이터 및 모델 아키텍처를 구축하며, 간단한 품질 검사 프로세스를 도입하는 데 집중해야 합니다. 또한, 팀원들의 지속적인 교육과 훈련을 통해 ML에 대한 이해도를 높이는 것이 중요합니다. 점진적으로 자동화와 확장성을 고려해야 합니다.

11 빠르게 훑어보기 (1페이지 요약)

CSCI103 Lecture 9 핵심 요약

1. MLOps의 본질:

- ML, 데이터 엔지니어링, 데브옵스의 융합.
- 모델 개발 → 배포 → 모니터링 → 재훈련의 연속적인 자동화.

2. 데이터의 중요성: '빅 굿 데이터':

- 데이터는 AI의 '음식'. 양(빅 데이터)보다 질(굿 데이터)이 중요.
- 데이터 중심 AI가 모델 중심 AI보다 성능 향상에 더 큰 영향.
- 좋은 데이터 = 명확한 레이블, 대표성, 시기적절한 피드백, 적절한 크기.

3. ML 프로젝트의 주요 역할:

- 비즈니스 이해관계자: 가치 창출, 문제 정의.
- 데이터 엔지니어: 데이터 파이프라인, 품질, 접근성.
- 데이터 과학자: ML 문제 전환, 피쳐, 모델 훈련/테스트.
- ML 엔지니어: 모델 배포, 운영, 재훈련 자동화.
- 데이터 거버넌스: 보안, 익명화, 규정 준수.

4. MLOps 3대 핵심 파이프라인:

- 훈련 파이프라인: 데이터 준비 → 피쳐 엔지니어링 → 모델 훈련 → 평가 → 등록.
- 추론 파이프라인: 새로운 데이터에 대한 예측 생성 (배치/실시간).
- 모니터링 파이프라인: 데이터/모델/개념 드리프트 감지, 성능 저하 알림, 자동 재훈련 트리거.

5. MLFlow의 역할:

- 트래킹: 실험 파라미터, 메트릭, 아티팩트, 코드, 데이터 기록.
- 레지스트리: 모델 버전 관리, 배포 단계 추적, 모델의 '진실의 원천'.
- 데이터 계보: 모델 훈련에 사용된 데이터셋 정보 기록 (추적성).

6. 모델 배포 및 자동화:

- 배포 전략: '모델 배포' vs. '코드 배포' (코드 배포가 더 정확하고 안정적).
- CI/CD: 지속적 통합/배포를 통한 자동화된 모델 배포 및 테스트.
- 하이퍼파라미터 튜닝: Optuna, Hyperopt 등 자동화된 방법으로 최적 모델 선정.
- 웹훅: 모델 환경 전환 시 자동 테스트, 알림 등 트리거.

7. MLOps 성숙도:

- 초급 → 중급 → 고급 단계로 발전.
- 고급 단계는 빠른 배포 주기, 엔드투엔드 자동화, 높은 복구력(Chaos Monkey, 롤백, DR) 특징.

8. 실습: 고객 이탈 예측 모델:

- 데이터 준비 → 피쳐 엔지니어링 → LightGBM 모델 훈련 → MLFlow 레지스트리 등록 → 챌린저 모델 검증 → 배치 추론.
- '챌린저' 모델이 '챔피언' 모델보다 우수하면 '챔피언'으로 승격.